

White-Label Design Token System

One Figma source of truth, many brands: compiled into type-safe Flutter code.

Flutter

Dart

Design Tokens

Code Generation

Figma Tokens Studio

Shell scripting

The problem

A single Flutter component library had to power several consumer brands, each with its own colours, typography, radii and light/dark variants. Design changes lived in Figma; copying hex values into Dart by hand was slow, error-prone and impossible to keep consistent across brands.

11

brand / theme variants

23

brand token sets in design
source

~18.7k

lines generated, 0 written by
hand

1

command to refresh all tokens

What I built

An automated pipeline that turns the design team's Figma token JSON into Dart, plus a tiny runtime layer that lets the whole app switch brand by swapping one object.

Figma tokens

Tokens Studio
JSON

→

Brand list

themes.json

→

Merge

one
tokens.json

→

Generator

figma2flutter

→

Dart classes

1 per brand

→

App

context.tokens

1 · Layered token architecture

Tokens are organised in layers, each overriding the previous. A brand only declares what makes it different.

Layer	Contains
Core	Raw palettes, sizing, spacing, radius, font weights
Alias	Meaning: accent, positive, negative, info; text styles
Theme	Light / Dark semantic roles: background, content, border, surface
Platform	App-specific shadows and borders
Brand	Small overrides, e.g. only the brand palette or font

2 · Brands as configuration

A brand is a few lines of JSON that select which layers are enabled. Adding one never touches Dart.

```
{ "name": "Fleetcor",
  "selectedTokenSets": {
    "01 Core": "source",    "02 Alias": "enabled",
    "03 Theme/Light": "enabled",
    "04 Platform/App": "enabled",
    "05 Brands/INF/Fleetcor": "enabled" } }    // -> class FleetcorTokens
```

Theme = Core + Alias + (Light | Dark) + Platform + Brand

3 · One script, reproducible output

A shell script downloads the latest tokens through the GitHub API, decodes them, injects our brand list with `jq`, and runs the generator. The merged `tokens.json` is committed, so every generated file can be traced back to its exact input.

```
curl ... tokensV2.json | jq -r '.content' | base64 --decode > unEditedTokens.json
jq '["$themes"] = input' unEditedTokens.json themes.json > tokens.json
figma2flutter --input tokens.json --output lib/tokens/
```

4 · Type-safe generated code

The generator resolves references and colour modifiers, then emits an abstract contract and one implementation per theme. The compiler guarantees every brand exposes exactly the same token names.

```
abstract class ITokens {
  ColorTokens get color;   SpacingTokens get spacing;
  TextStyleTokens get textStyle;  RadiiTokens get radii;
  ShadowTokens get shadow;  SizeTokens get size; ... }

class FleetcorTokens extends ITokens { ... }
class PBPDarkTokens extends ITokens { ... }
```

Type converters map each token type to its Flutter equivalent: `Color`, `MaterialColor`, `TextStyle`, `BorderRadius`, `EdgeInsets`, `BoxShadow`, `LinearGradient`. The generator is vendored in the repo, so it can be patched and versioned with the project.

5 · Zero-cost runtime brand switching

```
Tokens(
  tokens: FleetcorTokens(),
  child: MyApp(),
)
```

```
color: context.tokens.color.bgAccent
style: context.tokens.textStyle.labelSbold
gap:   context.tokens.spacing.spacingXs.left
```

The generated `Tokens` `InheritedWidget` and a `BuildContext` extension make tokens available anywhere. About seventy components read only semantic tokens, never a brand or a hex value, so changing the object re-skins the whole app and rebuilds dependants automatically. Light and dark are simply additional theme classes.

6 · Designer-friendly previews

A Widgetbook component gallery has a **Brand Theme** knob listing every generated brand, so designers and QA can review each component in each brand live, without running the product app.

Outcome

- **Design stays the source of truth:** no hand-typed hex values; a design update is one command and a reviewable diff.
- **Compile-time safety:** a missing or renamed token breaks the build, not production.
- **Fast onboarding:** a new brand is one JSON entry plus a regeneration.
- **Zero runtime parsing cost:** tokens are const Dart values.
- **Clean separation:** components are brand-agnostic and fully reusable.

Skills demonstrated

Design-system architecture · Design tokens & Figma integration · Code generation · Dart/Flutter `InheritedWidget` patterns · Build automation (bash, jq, curl) · Developer experience & tooling